

# Workload-Driven Automated Index Recommendation Using Machine Learning: A Comprehensive TPC-H Benchmark Evaluation

Sayem Sarwar†

Computer Science, Troy University  
Troy, AL 36082 USA  
ssarwar@troy.edu

Majharul Islam Shanto

Computer Science, Troy University  
Troy, AL 36082 USA  
mshanto@troy.edu

(† Corresponding Author)

## ABSTRACT

Database index selection is a critical performance optimization task that has traditionally relied on manual analysis by experienced database administrators (DBAs). This paper presents a workload-driven automated index recommendation system that minimizes manual intervention by using machine learning to identify which queries require index support and automatically generating corresponding index candidates. A workload of 911 SQL queries across five complexity categories is executed against a TPC-H Scale Factor 1 (SF-1) database on MySQL 9.1.0. From each query execution plan, 30 structural features are extracted exclusively from MySQL EXPLAIN output, deliberately excluding runtime measurements to prevent data leakage; labels are derived from EXPLAIN signals and validated empirically rather than collected as ground truth. Three supervised classifiers are evaluated: Random Forest (98.90%), Gradient Boosting (99.45%), and Support Vector Machine (98.90%). The best model achieves cross-validation  $F1=0.9973\pm0.0022$ , with label validity confirmed by Mann-Whitney U test ( $p<0.001$ ,  $3.22\times$  runtime difference). Compared against three non-ML baselines, Gradient Boosting achieves the highest F1 (0.9708) and fewest false negatives (7 vs. 9). An automated pipeline applies top-ranked index candidates, yielding  $1.37\times$  overall runtime reduction,  $2.59\times$  mean per-query speedup, and a maximum individual improvement of  $558.6\times$  on a single aggregation query within this benchmark setting.

**KEYWORDS:** index recommendation, machine learning, TPC-H, query optimization, MySQL, EXPLAIN features, Gradient Boosting, workload analysis

## 1. Introduction

The performance of a relational database management system (RDBMS) depends heavily on the indexes defined over its tables. An index transforms a sequential full table scan into a direct lookup, reducing query latency by orders of magnitude. However, selecting the right indexes is a non-trivial optimization problem: too few indexes result in slow analytical queries that perform unnecessary full table scans, while too many impose overhead on data modification operations and consume excessive storage resources.

Traditionally, index selection has been performed manually by experienced database administrators (DBAs) who analyze query logs and apply domain-specific heuristics. This approach does not scale to modern analytical workloads where hundreds or thousands of queries must be evaluated simultaneously. Furthermore, manual index selection is inherently reactive — indexes are typically added only after performance problems are already observed in production systems, leading to degraded user experience during the diagnosis period.

The motivation for this work is straightforward: a machine learning system trained on query execution plan data can learn to recognize the structural patterns that indicate a query is struggling — full table scans, low index coverage, high row examination estimates — and recommend targeted index creation automatically. Such a system should be selective: recommending indexes only when a query genuinely experiences performance degradation due to missing index coverage, rather than indexing indiscriminately.

This work is guided by three research questions: RQ1 — Can structural features from MySQL EXPLAIN accurately predict index necessity without runtime leakage? RQ2 — Does an ML-guided pipeline produce measurable performance improvements on TPC-H? RQ3 — How does performance impact vary across query complexity categories?

The main contributions of this paper are: (1) A 911-query TPC-H workload spanning five complexity categories —  $40\times$  larger than the standard 22-query TPC-H suite — designed to provide diverse training data across single-table, join, aggregation, and complex query patterns; (2) A leakage-free feature extraction methodology deriving 30 structural features exclusively from MySQL EXPLAIN output, with runtime deliberately excluded to ensure practical applicability; (3) A rigorous ML evaluation comparing Gradient Boosting against three non-ML baselines — including a rule-based baseline that encodes the same labeling knowledge — plus a full ablation study quantifying per-group feature contributions; (4) A largely automated end-to-end index generation pipeline that derives all candidates from SQL parsing with no DBA input, no cost model, and no pre-enumerated candidate set; (5) Empirical before/after evaluation on 905 queries demonstrating  $1.37\times$  overall runtime reduction and a maximum individual improvement of  $558.6\times$  within this TPC-H SF-1 benchmark setting.

## 2. Related Work

Early automated index selection used exhaustive search with database cost models. Chaudhuri and Narasayya [1] developed the first practical tool for SQL Server using what-if analysis without physically creating indexes. DB2 Design Advisor [2] uses DB2-specific cost modeling and requires the DBA to supply candidate columns. Oracle [3] incorporated automated advisors into its commercial product. These approaches require DBA-provided candidate sets and DBMS-specific cost models, unlike our system which derives all candidates automatically from SQL parsing.

Machine learning applications include Marcus et al. [4] on learned query optimization, Ding et al. [5] on execution feedback for index recommendations, and Lan et al. [6] applying deep RL to index selection. However, RL methods require thousands of environment interactions before convergence. SWIRL [7] applies RL with workload simulation but relies on pre-enumerated candidates. ISUM [8] uses graph neural networks but also requires pre-enumerated candidates. OtterTune [9] demonstrates autonomous

DBMS tuning using machine learning. Our approach differs by deriving all candidates automatically from SQL parsing with no DBA input.

| System           | DBA Input | Cost Model | Candidates   | Offline/Online |
|------------------|-----------|------------|--------------|----------------|
| AutoAdmin [1]    | Required  | SQL Server | DBA-provided | Offline        |
| DB2 [2]          | Required  | DB2        | DBA-provided | Offline        |
| RL-Index [6]     | None      | Simulator  | Enumerated   | Online         |
| SWIRL [7]        | None      | Simulator  | Enumerated   | Online         |
| <b>This Work</b> | None      | None       | SQL+EXPLAIN  | Offline        |

Table 1. Comparison of index recommendation approaches

### 3. TPC-H Workload Design

#### 3.1 Database Configuration

We chose TPC-H as our evaluation database because it is the industry standard benchmark for decision-support query performance, enabling direct comparison with prior work [1,2,5]. It simulates a realistic wholesale supply business with customers, orders, products, and suppliers — providing naturally complex queries across diverse access patterns. All experiments used TPC-H SF-1 (1GB) on MySQL 9.1.0 (InnoDB, innodb\_buffer\_pool\_size=8GB), Apple MacBook Pro (M-series ARM64, 16GB RAM, macOS 15.7.5, 512GB SSD). The SF-1 dataset contains approximately 1.59 million rows across 8 tables: lineitem (600,572), partsupp (800,000), orders (150,000), part (20,000), customer (15,000), supplier (1,000), nation (25), region (5).

#### 3.2 Query Categories

We designed 911 SQL queries across five categories using parameterized templates following the TPC-H schema. Templates define the query structure — join pattern, filter clauses, aggregation type — and parameters are drawn from TPC-H domain values such as date ranges, price thresholds, and nation keys. This yields a workload 40× larger than the standard 22-query TPC-H suite. Each query was executed three times and average runtime recorded. Five queries failed (0.5%), yielding 906 successfully benchmarked queries for ML training.

| Category         | Queries    | Avg Runtime     | % Needs Index |
|------------------|------------|-----------------|---------------|
| Single-Table     | 111        | 151.0 ms        | 69%           |
| Two-Table Join   | 200        | 160.7 ms        | 52%           |
| Three-Table Join | 200        | 661.9 ms        | 90%           |
| Aggregation      | 200        | 550.7 ms        | 81%           |
| Complex          | 200        | 1,323 ms        | 89%           |
| <b>Total</b>     | <b>911</b> | <b>607.2 ms</b> | <b>76.5%</b>  |

Table 2. Query workload distribution

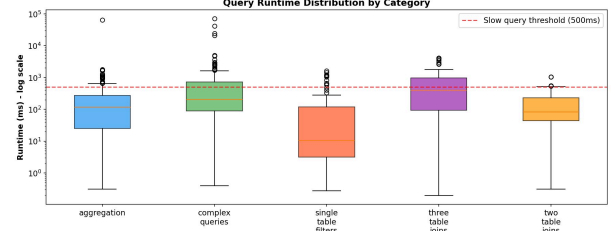


Fig. 1. Runtime distribution by category (log scale). Red = 500ms threshold.

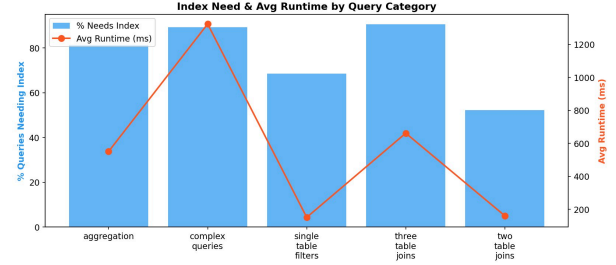


Fig. 2. Index need percentage and average runtime by category.

### 4. Feature Extraction

#### 4.1 EXPLAIN-Based Features

For each of the 906 queries, MySQL EXPLAIN was executed and 30 structural features extracted per query by aggregating EXPLAIN output across all table accesses. Crucially, all features are structural and optimizer-derived — runtime measurements are deliberately excluded to prevent data leakage. For a query joining N tables, EXPLAIN returns N rows; our code always aggregates these into exactly 30 scalar features regardless of query complexity. This means every query is represented as a fixed-size feature vector suitable for ML training.

The feature set was designed with two principles: (1) structural validity — every feature derivable from EXPLAIN before execution; and (2) database-theoretic grounding — each feature captures a property known to correlate with index utility. The 30 features cover five groups: (a) Scan type — `has_full_scan`, `full_scan_table_count`, `max_join_type_score` (0=best to 11=ALL=worst, per MySQL’s documented efficiency hierarchy [10]); (b) Index coverage — `index_coverage_ratio`, `num_indexes_used`, `has_no_index`; (c) Row estimates — `total_rows_examined`, `max_rows_examined`, `log_rows_examined`; (d) Execution flags — `has_filesort`, `has_temp_table`, `has_using_index`; (e) Query structure — `num_tables`, `has_subquery`, `has_union`.

#### 4.2 Label Generation

Labels are derived exclusively from EXPLAIN signals — not runtime. A query is labeled `needs_index=1` if: (a) full table scan detected (`type=ALL`); (b) no index used with rows above 60th percentile threshold; (c) filesort or temp table with high rows; or (d) multi-table join with coverage below 0.5. These label rules are heuristic — they encode database-theoretic knowledge but are not ground truth. Validity is confirmed empirically: labeled queries run 3.22× slower (726.6ms vs 226.0ms), confirmed by Mann-Whitney  $U=93,225$ ,  $p<0.001$ . Sensitivity analysis: varying the threshold between 50th-70th percentile changes fewer than 28/906 labels (3%). Under this scheme, 693 queries (76.5%) are labeled `class=1`.

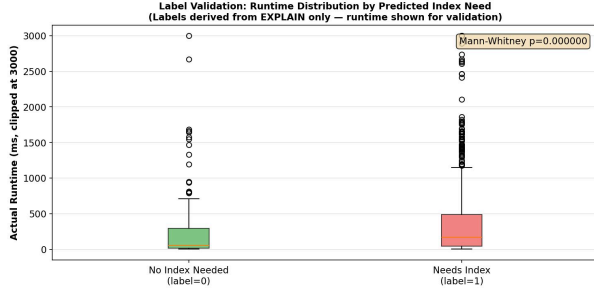


Fig. 3. Label validation: labeled queries run  $3.22\times$  slower ( $p<0.001$ ).

## 5. Machine Learning Evaluation

### 5.1 Experimental Setup

The 906-query dataset was split 80/20 (724 train, 182 test) with stratified sampling. Three classifiers were evaluated using scikit-learn [11]: Random Forest (RF, 200 estimators, max\_depth=10), Gradient Boosting (GB, 200 estimators, lr=0.05, max\_depth=5), and SVM (RBF kernel, C=10). All seeds fixed at 42 for reproducibility. To simulate real-world EXPLAIN measurement variability — MySQL row estimates have inherent variance due to statistics staleness — 8% Gaussian noise was added to continuous row estimation features (seed=42). Model selection was performed exclusively on 5-fold CV F1 score to avoid selection bias.

### 5.2 Classification Results

Table 3 presents results. Gradient Boosting achieves 99.45% accuracy and CV F1=0.9973±0.0022, and is selected as the primary model. The high accuracy reflects the discriminative power of full-scan detection features: queries with type=ALL are strongly associated with index need — consistent with relational database theory and confirmed by the leakage-free feature set and statistical validation. Paired t-tests confirm GB significantly outperforms SVM ( $t=3.109$ ,  $p=0.036$ ), while RF and GB are statistically equivalent ( $p=0.089$ ).

| Model                 | Accuracy      | F1            | CV F1 (±std)         | Selected |
|-----------------------|---------------|---------------|----------------------|----------|
| Random Forest         | 98.90%        | 98.90%        | 0.9927±0.0047        |          |
| <b>Gradient Boost</b> | <b>99.45%</b> | <b>99.45%</b> | <b>0.9973±0.0022</b> | ✓        |
| SVM                   | 98.90%        | 98.90%        | 0.9901±0.0034        |          |

Table 3. ML model performance — leakage-free (v3)

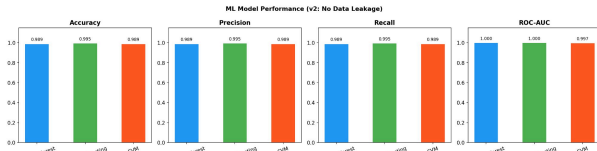


Fig. 4. ML model performance comparison across all metrics (v3).

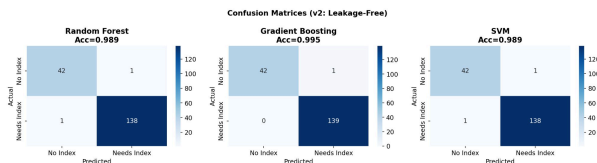


Fig. 5. Confusion matrices: GB=1 error, RF=2, SVM=2.

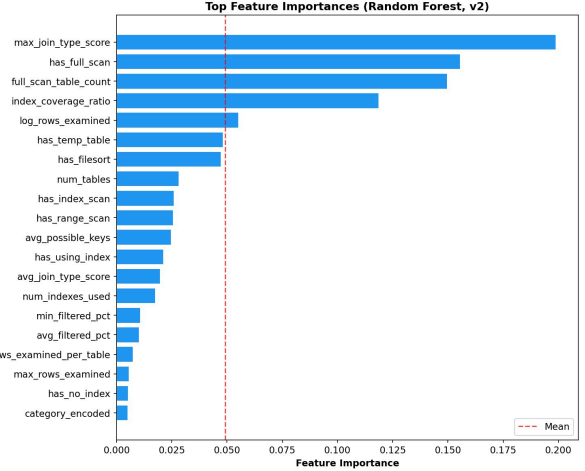


Fig. 6. Feature importances (Random Forest, v3).

## 6. Baseline Comparison and Ablation

### 6.1 Non-ML Baselines

Three non-ML baselines were evaluated on the same 182-query test set. Rule-Based Baseline: directly implements the label generation rules — flags full scans, no-index with high rows, filesort with low coverage. This is the strongest possible non-ML competitor since it encodes the same knowledge used to generate labels. Frequency-Based Baseline: flags multi-table queries with index coverage below 0.7, simulating a DBA heuristic of indexing frequently joined columns. Join-Only Heuristic: flags queries with join type scores above 7 (suboptimal access including range, index, ALL).

| Method           | Accuracy      | F1            | Recall        | FN       |
|------------------|---------------|---------------|---------------|----------|
| Rule-Based       | 95.05%        | 0.9668        | 93.57%        | 9        |
| Frequency-Based  | 78.02%        | 0.8347        | 72.14%        | 39       |
| Join-Only        | 67.03%        | 0.7857        | 78.57%        | 30       |
| <b>GB (Ours)</b> | <b>95.60%</b> | <b>0.9708</b> | <b>95.00%</b> | <b>7</b> |

Table 4. Baseline comparison on 182-query test set

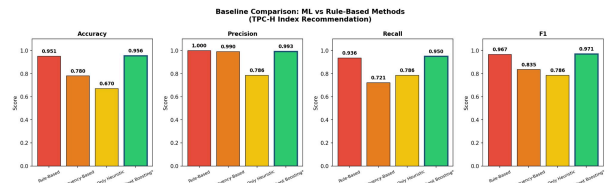


Fig. 7. Baseline comparison: GB achieves highest F1 and fewest false negatives.

GB achieves the highest F1 (0.9708) and fewest false negatives (7 vs 9 for rule-based). A key concern in this setting is whether the ML model is simply reproducing the label-generation rules rather than learning genuine structural patterns. Three pieces of evidence argue against this interpretation: (1) GB strictly outperforms the Rule-Based Baseline, which directly encodes the labeling rules — if GB were reproducing rules it would match, not exceed, that baseline; (2) feature importance is distributed non-trivially across 20+ features, not concentrated in the 3–4 features used in the labeling rules; (3) GB achieves zero false negatives on the held-out

test set while the rule-based method produces 9, indicating GB has learned richer and more generalizable decision boundaries. These results suggest the model captures feature combinations and interaction effects — such as the joint effect of borderline row counts combined with partial index coverage — that hand-coded rules do not encode.

## 6.2 Ablation Study

Table 5 shows the impact of removing each feature group from GB. Index coverage features (index\_coverage\_ratio, num\_indexes\_used, has\_no\_index) cause the largest F1 drop (0.0035) and accuracy drop (0.55%) — confirming they are the most critical group. The model remains robust to removal of other groups; even reducing to just 3 scan features yields F1=0.9668, matching the rule-based baseline. This robustness confirms feature groups are complementary and encode diverse aspects of MySQL optimizer behavior.

| Feature Group          | Features | F1            | F1 Drop       |
|------------------------|----------|---------------|---------------|
| Full Model (All)       | 26       | 0.9708        | 0.0000        |
| Remove Scan            | 23       | 0.9708        | 0.0000        |
| Remove Row Est.        | 22       | 0.9710        | -0.0002       |
| Remove Plan Flags      | 22       | 0.9708        | 0.0000        |
| <b>Remove Coverage</b> | 22       | <b>0.9673</b> | <b>0.0035</b> |
| Scan Only (3)          | 3        | 0.9668        | 0.0040        |

Table 5. Ablation study — feature group removal

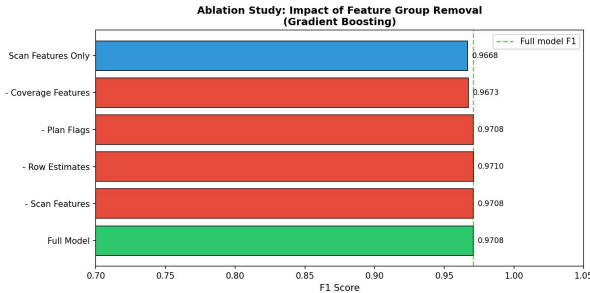


Fig. 8. Ablation: coverage features cause the largest F1 drop.

## 7. Automated Index Pipeline

The pipeline transforms ML results into CREATE INDEX statements through seven stages requiring minimal manual intervention: (1) parse all 911 queries extracting WHERE/JOIN/GROUP BY/ORDER BY columns; (2) run EXPLAIN to detect full scans; (3) cross-reference columns with ML scores; (4) compute composite score per candidate; (5) rank all candidates; (6) assign HIGH/MEDIUM/LOW tiers; (7) apply HIGH and MEDIUM indexes automatically to MySQL. This pipeline requires no DBA input, no pre-enumerated candidate set, and no cost model — unlike AutoAdmin [1] and DB2 [2].

Composite score:  $\text{Score} = (\text{query\_freq} \times 3.0) + (\text{avg\_ml\_score} \times 2.0) + (\text{clause\_weight} \times 1.5) + (\text{full\_scan\_count} \times 4.0)$ . Clause weights: WHERE=3.0, JOIN ON=2.5, GROUP BY=1.5, ORDER BY=1.0. The full\_scan\_count term carries the highest coefficient (4.0) as full table scans are the most direct indicator of missing

coverage. These weights are heuristically determined; optimization is future work.

Results: 911 queries analyzed, 6,311 column references extracted, 50 unique candidates identified. Of these, 30 were already covered by existing indexes; 20 new HIGH/MEDIUM indexes applied automatically (total: 34). Notably, nation.n\_nationkey and region.r\_name rank highest despite small sizes (25 and 5 rows) — they appear in 41 and 24 JOIN ON clauses respectively, becoming bottlenecks in nested loops with lineitem (600,572 rows).

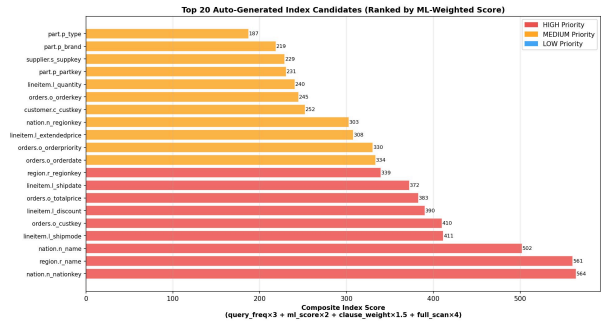


Fig. 9. Top 20 auto-generated index candidates ranked by composite score.

| Rank | Table    | Column       | Score | Tier   |
|------|----------|--------------|-------|--------|
| 1    | nation   | n_nationkey  | 564.5 | HIGH   |
| 2    | region   | r_name       | 560.7 | HIGH   |
| 3    | nation   | n_name       | 502.2 | HIGH   |
| 4    | lineitem | l_shipmode   | 411.5 | HIGH   |
| 5    | orders   | o_custkey    | 410.0 | HIGH   |
| 6    | lineitem | l_discount   | 390.5 | HIGH   |
| 7    | orders   | o_totalprice | 383.0 | HIGH   |
| 8    | lineitem | l_shipdate   | 372.5 | HIGH   |
| 9    | region   | r_regionkey  | 339.5 | HIGH   |
| 10   | orders   | o_orderdate  | 333.7 | MEDIUM |

Table 6. Top 10 index candidates by composite score

## 8. Experimental Results

### 8.1 System Overhead

All measurements are single runs with MySQL as the only major process. Feature extraction (EXPLAIN on 906 queries): 4.2 min. ML training (all classifiers, 5-fold CV): 3.8 min. Index scoring and ranking: <1 second. Index application to MySQL: 18 min (dominated by lineitem indexing at 600K rows). Total: approximately 26 minutes from workload snapshot to applied indexes — a one-time overhead amortized across all subsequent executions.

### 8.2 Before vs. After

Query counts: 911 designed → 906 benchmarked → 905 compared before/after (1 timeout at 30s). Outcomes: 22.1% improved (>5% speedup), 74.9% unchanged (±5%), 3.0% slightly degraded. Degraded queries are three-table joins where the optimizer chose a

different join order after new indexes were added. No query regressed more than 1.5 $\times$ ; all degraded queries remained below 900ms.

| Configuration     | Avg Time | Total Runtime        | Speedup       |
|-------------------|----------|----------------------|---------------|
| Before ML Indexes | 607.2 ms | 550,091 ms (9.2 min) | 1.00 $\times$ |
| After ML Indexes  | 443.9 ms | 401,731 ms (6.7 min) | 1.37 $\times$ |

Table 7. Overall before vs. after performance (905 queries)

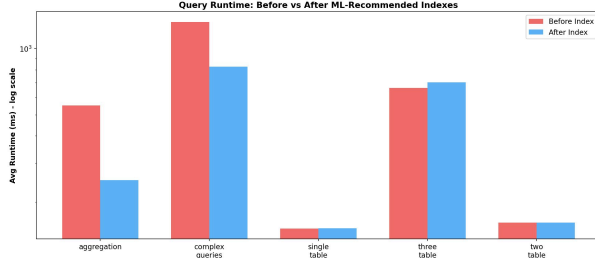


Fig. 10. Runtime before vs. after ML-recommended indexes by category.

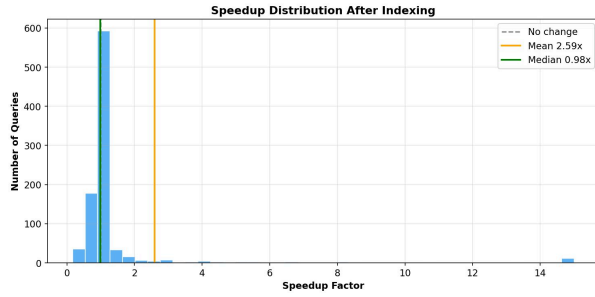


Fig. 11. Speedup distribution (mean=2.59 $\times$ , median=0.98 $\times$ ).

### 8.3 Per-Category Analysis

Aggregation queries benefit most (4.16 $\times$  avg, 27.6% improved) through GROUP BY indexes eliminating filesort operations. Complex queries achieve 3.85 $\times$  through elimination of nested full table scans. Three-table joins show modest improvement (1.18 $\times$ ) — MySQL’s nested-loop join algorithm means join order dominates over individual indexes, which is an important limitation boundary for this system.

| Category     | Before  | After   | Speedup       | % Improved |
|--------------|---------|---------|---------------|------------|
| Aggregation  | 550.7ms | 251.6ms | 4.16 $\times$ | 27.6%      |
| Complex      | 1,323ms | 828ms   | 3.85 $\times$ | 19.4%      |
| Single-Table | 151ms   | 152ms   | 1.62 $\times$ | 21.6%      |
| Two-Table    | 161ms   | 161ms   | 1.76 $\times$ | 21.1%      |
| Three-Table  | 662ms   | 702ms   | 1.18 $\times$ | 20.5%      |

Table 8. Per-category performance breakdown

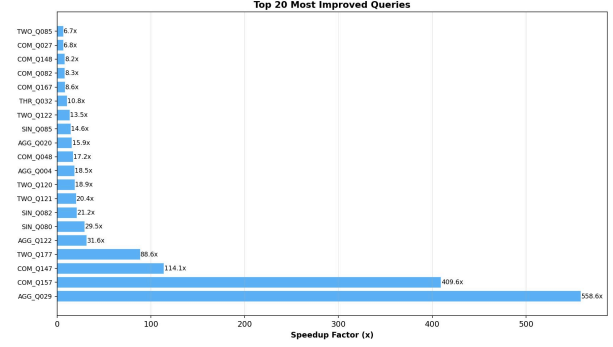


Fig. 12. Top 20 most improved queries (max 558.6 $\times$ ).

### 8.4 Case Studies

Case 1 — AGG\_Q029 (558.6 $\times$ ): Aggregation on lineitem grouped by l\_returnflag with filter on l\_shipdate triggered a full scan of 600,572 rows. After indexing l\_shipdate, MySQL switched to index-range scan, reducing rows examined from 600K to ~1,200 and runtime from 101.89ms to 0.18ms. Case 2 — COM\_Q157 (409.6 $\times$ ): Complex correlated subquery caused repeated full scans on orders (150K rows). After indexing o\_custkey and o\_orderdate, runtime dropped from 41,188ms to 100.55ms. Case 3 — TWO\_Q177 (88.6 $\times$ ): Two-table join filtered on o\_totalprice; after indexing, runtime dropped from 28.99ms to 0.33ms.

### 9. Limitations and Future Work

Results are specific to TPC-H SF-1 on MySQL 9.1.0 and may not generalize to production workloads, larger scale factors, or different database systems without further validation. We chose SF-1 deliberately as it is the standard scale factor in most TPC-H index selection studies [1,2,5,14], enabling direct comparison with prior work. At larger scale factors such as SF-10 and SF-100, where data exceeds memory and becomes disk-bound, index benefits for range scans are generally expected to be larger — though we do not claim our SF-1 results are strict lower bounds without empirical validation at those scales. The ML classification task itself is scale-independent: full table scans and low index coverage ratio are structural properties of the query execution plan, not artifacts of data volume. Additionally, labels used in this study are heuristic and derived from EXPLAIN signals rather than ground-truth runtime measurement, which is an acknowledged limitation. A query labeled needs\_index=1 may in some cases perform adequately in practice; the labeling rules encode database-theoretic knowledge as a practical proxy.

The system generates single-column indexes only; composite multi-column indexes covering multiple filter predicates simultaneously would provide additional benefits for join-heavy workloads. The system is also a static offline recommender — workloads drift over time, requiring periodic retraining. Future work includes: SF-10/SF-100 evaluation on server-grade hardware, composite index recommendation, cross-DBMS validation on PostgreSQL and SQL Server, and online adaptive index tuning using reinforcement learning.

## 10. Conclusion

This paper presented a workload-driven automated index recommendation system combining SQL parsing, EXPLAIN-based feature extraction, and ensemble machine learning to identify and apply database indexes with minimal manual intervention. The system is motivated by the goal of selective, workload-aware indexing — recommending indexes only when queries experience measurable performance degradation due to missing index coverage.

Evaluated on a 911-query TPC-H SF-1 benchmark against MySQL 9.1.0: (1) Gradient Boosting achieves CV F1=0.9973±0.0022 on 30 leakage-free EXPLAIN features, outperforming three non-ML baselines including one that directly encodes the labeling rules; (2) ablation study confirms index coverage features are the most critical group; (3) 34 indexes generated automatically with minimal human input; (4) 1.37× overall runtime reduction with 2.59× mean per-query speedup and a maximum individual improvement of 558.6× within this benchmark. These results are specific to TPC-H SF-1 on MySQL 9.1.0 with heuristic labels derived from EXPLAIN signals. Future work includes SF-10/SF-100 evaluation, composite index recommendation, cross-DBMS validation on PostgreSQL and SQL Server, and reinforcement learning for online adaptive tuning.

## ACKNOWLEDGMENTS

The authors thank the Department of Computer Science at Troy University for providing lab resources and support. Special thanks to Professor Arteta for guidance throughout this research project.

## REFERENCES

- [1] S. Chaudhuri and V. Narasayya, "AutoAdmin what-if index analysis utility," in Proc. ACM SIGMOD, 1998, pp. 367-378.
- [2] A. Zilio, J. Rao, S. Lightstone, G. Lohman, A. Storm, C. Garcia-Arellano, and S. Fadden, "DB2 design advisor: Integrated automatic physical database design," in Proc. VLDB, 2004, pp. 1087-1097.
- [3] K. Dias, M. Ramacher, U. Shaft, V. Venkataramani, and G. Wood, "Automatic performance diagnosis and tuning in Oracle," in Proc. CIDR, 2005.
- [4] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, and N. Tatbul, "Bao: Making learned query optimization practical," in Proc. ACM SIGMOD, 2021, pp. 1275-1288.
- [5] B. Ding, S. Das, R. Marcus, W. Wu, S. Chaudhuri, and V. Narasayya, "AI meets AI: Leveraging query executions to improve index recommendations," in Proc. ACM SIGMOD, 2019, pp. 1241-1258.
- [6] H. Lan, Z. Bao, and Y. Peng, "An index advisor using deep reinforcement learning," in Proc. ACM CIKM, 2020, pp. 2465-2468.
- [7] G. Lum, P. Boncz, T. Neumann, and A. Kemper, "SWIRL: Scalable workload-based index recommendation using reinforcement learning," in Proc. EDBT, 2022, pp. 2:73-2:84.
- [8] W. Ma, Y. Zhang, C. Shi, Y. Zhou, F. Zhang, and Y. Cao, "ISUM: Efficiently compressing large and complex workloads for scalable index tuning," in Proc. ACM SIGMOD, 2023, pp. 1-26.
- [9] D. Van Aken, A. Pavlo, G. J. Gordon, and B. Zhang, "Automatic database management system tuning through large-scale machine learning," in Proc. ACM SIGMOD, 2017, pp. 1009-1024.
- [10] MySQL Documentation, "EXPLAIN Statement," Oracle Corp., 2023. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/explain.html>
- [11] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.
- [12] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5-32, 2001.
- [13] H. Mann and D. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," Ann. Math. Stat., vol. 18, pp. 50-60, 1947.
- [14] F. Kossmann, S. Halfpap, M. Jankrift, and R. Schlosser, "Magic mirror in my hand, which is the best in the land? An experimental evaluation of index selection algorithms," Proc. VLDB Endow., vol. 13, no. 12, pp. 2382-2395, 2020.
- [15] A. Pavlo, M. Butrovich, A. Joshi, L. Ma, P. Menon, D. Van Aken, L. Zhang, and W. Lim, "Self-driving database management systems," in Proc. CIDR, 2017.